



30-Day Android Challenge

Mastering Android Programming Interviews Globally
(Junior-Mid-Senior-Architect-Level Deconstruction 📱)

Day 25:

Modularization & Dynamic Feature Modules



Esmaeetil Moradi

@esmaeetil-moradi700

Day 25



of #30DayAndroidChallenge:

Comprehensive Answers for "Day 25: Modularization & Dynamic Feature Modules" - #30DayAndroidChallenge

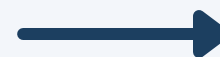
Hello to all my dear friends and companions! 🙌

Welcome to Day 25 of the #30DayAndroidChallenge! Today, we're diving into one of the most powerful concepts in modern Android development: Modularization. This process is crucial for building large, scalable, and maintainable applications. It not only helps with code organization but also has a direct impact on build times and user experience.



Esmaeeil Moradi

@esmaeeil-moradi700



Junior Level Questions



Esmaeeil Moradi
@esmaeeil-moradi700



What is app modularization and what are its benefits?

Modularization is the process of breaking down a large, monolithic application into several smaller, independent, and manageable modules. Each module can represent a specific feature, a layer of the architecture (like the data layer), or a common library.



Esmaeel Moradi
@esmaeeil-moradi700



Modularization offers several benefits:

1. **Improved Build Times:** This is the biggest advantage. When you change only a small module, only that module and its dependent modules are recompiled, not the entire project. This significantly reduces build times, especially in large projects.
2. **Better Code Organization:** Each module creates a clear boundary for a feature or an architectural layer. This makes the code cleaner, easier to understand, and prevents unnecessary complexity.
3. **Code Reusability:** Modules can act as independent libraries. A UI module or a network module can be reused across multiple applications or in multiple parts of the same application.
4. **Facilitated Teamwork:** Different teams can work on different modules in parallel without worrying about creating conflicts in each other's code.
5. **Easier Dependency Management:** Modularization allows you to manage dependencies more precisely and prevent hidden dependencies.



Esmaeel Moradi

@esmaeel-moradi700



In short, modularization helps you break down a large project into manageable parts and work on it faster, in a more organized, and more efficient way.



Esmaeeil Moradi
@esmaeeil-moradi700



Explain dynamic feature modules and on-demand delivery.

Dynamic Feature Modules are a special type of module introduced by the Android App Bundle. These modules allow you to deliver specific features and functionalities of your app separately from the base app.

The key difference from regular modules is:

- Regular modules are installed with the base app at the initial installation.
- Dynamic feature modules are not included in the initial install and can be downloaded "on-demand," meaning only when the user needs that feature.



Esmaeel Moradi

@esmaeeil-moradi700



On-demand delivery is the concept that enables the download of dynamic feature modules. This has several advantages:

- **Reduced Initial App Size:** Users only download the base app with the minimum essential features. This makes the app download and install faster.
- **Improved User Experience:** Users can download the features they need when they need them. For example, if a chat app has a video call feature, this feature can be designed as a dynamic module so that only users interested in that feature download it.
- **Optimized Resource Usage:** System resources, like storage space, are only consumed by features that are actually used.



Esmaeel Moradi

@esmaeel-moradi700



This approach is ideal for apps with large features that not all users need and directly improves user experience and increases install rates.



Esmaeeil Moradi
@esmaeeil-moradi700

How do you structure a modular Android application?



The structure of a modular Android application typically follows a layered or feature-based design. The most common structure is as follows:



Esmaeeil Moradi
@esmaeeil-moradi700



- **:app Module:** This is the entry point of your application and the main module used to produce the final file (APK or AAB). This module depends on all feature modules but should not contain business logic or data. Its role is to coordinate all modules.
- **:core or :common Module:** This module contains shared, common code that is used throughout the app. For example:
 - Utility classes.
 - Common UI components (like custom buttons or dialogs).
 - Shared constants and utilities. This module is a dependency for all other modules.
- **:data Modules:** These modules contain the app's data layer. Each :data module can include:
 - **Network Libraries:** API clients and their responses.
 - **Database:** Logic related to the local database (like Room).
 - **Repositories:** Logic for fetching data from different sources and providing it to the layer above.
- **:feature_X Modules:** These are feature-based modules, and each one implements a specific app feature. For example, **feature_login**, **feature_profile**, or **feature_chat**. Each feature module typically depends on the **:data** and **:core** modules.



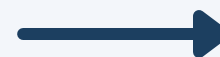
Esmaeeil Moradi
@esmaeeil-moradi700



This structure creates a unidirectional dependency flow (:app -> :feature -> :data -> :core) and prevents circular dependencies. This approach allows teams to work on different features in parallel and maximize build optimizations.



Esmaeeil Moradi
@esmaeeil-moradi700



Mid-Level Questions



Esmaeel Moradi
@esmaeeil-moradi700



Implement dynamic feature modules with complex dependencies.

Implementing dynamic feature modules with complex dependencies requires careful management of dependencies in the build.gradle.kts files. A common scenario is a dynamic feature module depending on shared code in the base module or another dynamic feature module.



Esmaeel Moradi
@esmaeel-moradi700



Gradle File Structure:

1-:app Module (Base Module): This module must declare the dynamic modules using dynamicFeatures in its build.gradle.kts file.

```
// File: app/build.gradle.kts
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    // ...
    dynamicFeatures += setOf(":dynamic_feature_video")
}
```



Esmaeel Moradi
@esmaeeil-moradi700



2.Dynamic Feature Module (:dynamic_feature_video): This module must apply the com.android.dynamic-feature plugin and depend on the base module. The dependency on the base module is defined using implementation(project(":app")).

```
// File: dynamic_feature_video/build.gradle.kts
plugins {
    id("com.android.dynamic-feature")
    id("org.jetbrains.kotlin.android")
}

android {
    // ...
}

dependencies {
    // A dynamic feature module must depend on the base module.
    // This is a special dependency that allows the dynamic feature
    // to access classes from the base app.
    implementation(project(":app"))

    // Add dependencies on shared core libraries or data modules.
    implementation(project(":core_ui"))
    implementation(project(":data_video"))
}
```



Esmaeel Moradi
@esmaeeil-moradi700



Implementing Dependencies on Other Modules:

- **Dependency on Regular Modules:** A dynamic feature module can depend on any regular library module (like `:core_ui` or `:data_video`). These regular modules are downloaded with the base app during the initial install.
- **Dependency on Each Other:** If one dynamic feature module (`feature_video_call`) depends on another (`feature_video_player`), this dependency is defined in `build.gradle.kts` as `implementation(project(":dynamic_feature_video_player"))`. Important Note: If `feature_video_call` is requested, the Play Core Library will automatically download `feature_video_player` as well. This simplifies the management of inter-feature dependencies.



Esmaeel Moradi
@esmaeel-moradi700



This approach ensures that all necessary dependencies are available before the dynamic feature module's code is accessed.



Esmaeel Moradi
@esmaeeil-moradi700



How do you handle shared resources across modules?

Managing resources (like strings, colors, drawables, and dimensions) in a modular app is crucial to avoid duplication and maintain design consistency. There are two main strategies for this:



Esmaeeil Moradi
@esmaeeil-moradi700



1. Using a Shared Resources Module:

This is the most common and best solution. You create a separate library module (e.g., **:core_res** or **:core_ui**) that contains all shared resources. Structure:

- In this module, you place strings.xml, colors.xml, dims.xml, and themes.xml files.
- All other modules (both feature modules and others) depend on this module.
- **Benefits:**
 - **No Duplication:** Resources are defined in only one place.
 - **Consistency:** Changing a color or a string will apply to all modules.
 - **Better Organization:** All UI resources are in a central location.



Esmaeel Moradi
@esmaeel-moradi700



2.Using Gradle Properties:

You can define resources in Gradle files and inject them into the project at build time. This method is more often used for variable values like API keys.

Example:

```
// File: build.gradle.kts
android {
    buildTypes {
        release {
            resValue("string", "api_key", "${System.getenv("API_KEY")}")
        }
    }
}
```

This method is less used for strings and colors because the shared resources module approach makes them easier to manage and edit in Android Studio.



Esmaeel Moradi
@esmaeel-moradi700



Using a shared resources module is a simple yet powerful way to ensure consistency and reduce human error in modular projects.



Esmaeel Moradi
@esmaeeil-moradi700



Design module communication strategies.

Communication between modules must be designed in a way that maintains loose coupling between them. A module should not directly call classes from another module, as this creates tight coupling and can lead to circular dependencies.

There are three main strategies for module communication:



Esmaeel Moradi
@esmaeeil-moradi700



1. Using Interfaces and Inversion of Control (IoC):

This is a popular design pattern.

- **Implementation:** A feature module (e.g., :feature_login) defines an interface within its module. This interface specifies the operations it needs.
- **Communication:** Another module (e.g., :app) implements this interface and makes it available to the feature module using a Dependency Injection (DI) framework like Hilt or Koin.
- **Benefit:** The feature module only depends on the interface it defined, not the actual implementation. This keeps the modules completely independent.



Esmaeel Moradi

@esmaeel-moradi700



2.Using a Central Router or Navigator:

This pattern is used for communication between UI modules.

- **Implementation:** An :app or :core_navigation module manages a central router.
- **Communication:** To navigate to another module, a feature module instructs the router to go to a specific destination instead of directly calling the other module. The router identifies the destination via a URI or a specific address and performs the navigation.
- **Benefit:** This removes the dependency between UI modules and keeps them fully independent.



Esmaeel Moradi

@esmaeel-moradi700



3.Using Events and Pub/Sub:

This pattern is used for asynchronous communication between modules.

- **Implementation:** Use an event library like Kotlin Flow or a Pub/Sub Bus.
- **Communication:** One module "publishes" an event (e.g., UserLoggedInEvent). Any other module that "subscribes" to that event receives and processes it.
- **Benefit:** This pattern completely removes the dependency between the publisher and the subscriber and is ideal for scenarios where multiple parts of the app need to react to a single event.



Esmaeel Moradi

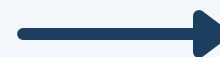
@esmaeeil-moradi700



For most projects, a combination of Interfaces (for business logic) and a Central Router (for navigation) is the best solution.



Esmaeel Moradi
@esmaeeil-moradi700



Senior Level Questions



Esmaeeil Moradi
@esmaeeil-moradi700



Create a comprehensive modularization strategy for large applications.

For large-scale applications, a comprehensive modularization strategy goes beyond a simple structure. You must combine a layered or feature-based architecture with strict rules for dependency and data flow.



Esmaeail Moradi
@esmaeail-moradi700



1. Hybrid Architecture:

- **Layering:** Modules should be classified based on their architectural layer (e.g., data, domain, core). These layers must follow a unidirectional dependency flow.
- **Feature-Based:** Within these layers, modules should be divided based on features (e.g., feature_auth, feature_profile). A feature module should only depend on modules from the layers below it.
- **Changeability and Stability:** Modules in the lower layers (like data, core) should be more stable and have fewer dependencies, while feature modules in the higher layers can have more frequent changes.

2. Managing Dependencies with a Dependency Graph:

- **Dependency Graph:** For large projects, having a visual dependency graph of all modules is crucial. It visually shows dependencies and helps in identifying circular or forbidden dependencies.
- **Enforce Unidirectional Flow:** You can use Gradle plugins or custom scripts to enforce rules that prevent circular dependencies or a data module from depending on a UI module.



Esmaeel Moradi

@esmaeel-moradi700



3. Common UI Modules:

- To maintain design consistency throughout the app, create a shared UI module (e.g., :core_ui) that contains all common UI components (like buttons, cards, styles). This module should only depend on UI libraries like Jetpack Compose or Views and have no business logic.

4. Using Build Source Modules:

- To centrally manage dependencies and versions, use Gradle Version Catalogs or a buildSrc module. This ensures that all teams use the same versions of libraries and makes updates easy to manage.



Esmaeel Moradi

@esmaeel-moradi700



A comprehensive modularization strategy is not just about splitting code; it's a structure, a dependency flow, and a set of tools and rules that allow teams to work efficiently and in a coordinated manner on a large codebase.



Esmaeel Moradi
@esmaeeil-moradi700



Implement dynamic module loading with performance optimization.

Implementing dynamic module loading requires advanced strategies to optimize performance and user experience. On-demand delivery is great, but you don't want to make the user wait for a module to download.



Esmaeel Moradi
@esmaeel-moradi700



1. Prefetching based on user behavior:

- **Concept:** Instead of waiting for the user to need a feature, prefetch its module in the background when it's likely the user will need it.
- **Implementation:** Use analytics tools (like Firebase Analytics) to track user paths. For example, if users frequently navigate from the main page to the "Settings" page, you can start downloading the "Settings" module in the background when the user opens the main page.

2. Conditional Delivery:

- **Concept:** You can define conditions for a module's download, such as device type, OS version, or geographical region.
- **Implementation:** Use the App Bundle API in the build.gradle.kts file to specify that a module should only be downloaded for devices with specific capabilities (like AR).



Esmaeel Moradi

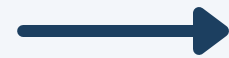
@esmaeel-moradi700



```
// File: dynamic_feature_ar/build.gradle.kts
android {
    // ...
    bundle {
        // A dynamic feature module that is only downloaded on AR-compatible devices.
        density { enableSplit = false }
        abi { enableSplit = false }
        language { enableSplit = false }
        texture { enableSplit = false }
        // The magic is in the manifest file
    }
}
```



Esmaeel Moradi
@esmaeel-moradi700



3. Manage Loading State:

- While a module is loading, you must show the user the progress (e.g., a progress bar).
- You must also handle error states, such as when the download fails or there's not enough space on the device.

4. Binary Code and Resources:

- Only place the code and resources that are absolutely necessary for that feature in dynamic modules.
- Place large binary files (like AI models, video files) in dynamic modules to reduce the base app size.



Esmaeel Moradi
@esmaeel-moradi700



These optimizations allow you to benefit from on-demand delivery while providing a smooth and uninterrupted user experience.



Esmaeeil Moradi
@esmaeeil-moradi700



How would you manage module dependencies and versioning?

Managing dependencies and versioning in large projects is a big challenge due to the high number of modules and libraries. Decentralized dependency management can lead to circular dependencies, version conflicts, and slow build times.



Esmaeeil Moradi
@esmaeeil-moradi700



1. Using Gradle Version Catalogs (Recommended Approach):

- **Concept:** Gradle Version Catalogs are a modern, built-in solution for centralizing dependency management. They allow you to define all versions and libraries in a single, central file.
- **Implementation:**
 - Create a `gradle/libs.versions.toml` file in the project root.
 - Define all versions, libraries, and bundles in this file.
 - In each module's `build.gradle.kts` file, reference the catalog instead of using "magic strings."
- **Benefit:** This approach is type-safe, meaning you'll get a compile error if you misspell a name. It also makes updating dependencies very easy.



Esmaeel Moradi
@esmaeel-moradi700



```
# File: gradle/libs.versions.toml
[versions]
androidx-core = "1.13.1"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "androidx-core" }
```

```
// File: app/build.gradle.kts
dependencies {
    implementation(libs.androidx.core.ktx)
}
```



Esmaeel Moradi
@esmaeeil-moradi700

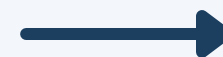
2. Using the buildSrc Module:

- **Concept:** buildSrc is a special Gradle module that allows you to write your build code in Kotlin or Groovy.
- **Implementation:** Create a buildSrc directory in the project root and place Kotlin files in it to define dependencies and versions.
- **Benefit:** This method is very powerful and allows for complex logic in the build. However, Gradle Version Catalogs are often the better option due to their simplicity and official support.

```
// File: buildSrc/src/main/kotlin/Dependencies.kt
object Versions {
    const val KOTLIN = "1.9.23"
    const val HILT = "2.51"
}
```



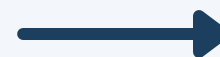
Esmaeel Moradi
@esmaeeil-moradi700



By using these tools, you can ensure version consistency across the project and significantly improve the maintenance process.



Esmaeel Moradi
@esmaeeil-moradi700



Architect Level Questions



Esmaeeil Moradi
@esmaeeil-moradi700



Architect a modular system that supports hundreds of feature modules.

Architecting a modular system for hundreds of modules goes beyond a simple structure and comes with complex challenges. The main focus must be on managing scale, build performance, and governance.



Esmaeel Moradi
@esmaeeil-moradi700



1. Strict Module Governance:

- **Dependency Rules:** You must enforce strict rules for dependencies. For example, a feature module should not depend on another feature module. These rules should be automatically checked in the CI/CD pipeline using tools like Dependency-Check or custom Gradle scripts.
- **Module Ownership:** Each module must have a responsible team or individual. This helps in maintaining code quality and clarity.

2. Build Performance Optimization:

- **Remote Build Caching:** This is the most crucial tool for managing hundreds of modules. By using a remote cache, if one team in Europe builds a module, another team in Asia can reuse the cached output.
- **Sharding:** Build and test tasks must be split across multiple agents or runners in the CI/CD pipeline to run in parallel.



Esmaeel Moradi

@esmaeel-moradi700



3. Managing Complexity with Tools:

- **Automation Tools:** Use Gradle scripts or custom plugins to automate repetitive processes, such as creating new modules.
- **Visualization Tools:** Use tools to visualize the dependency graph to understand dependencies and module boundaries.
- **Code Searchability:** With a large number of modules, code search becomes harder. Tools like SourceGraph or IDE-level search help teams.

4. Centralized Dependency Management:

- Using Gradle Version Catalogs or a buildSrc module is essential at this scale.
- With a centralized system, you can easily update a library in hundreds of modules.



Esmaeel Moradi

@esmaeeil-moradi700



This architecture manages complexity at the infrastructure level, allowing teams to focus on implementing features instead of build issues.



Esmaeel Moradi
@esmaeeil-moradi700



Design module governance and development workflows for large teams.

Module governance is a set of rules, processes, and tools that define how modules are created, maintained, and used in a large organization.



Esmaeeil Moradi
@esmaeeil-moradi700



1. Module Standardization:

- **Module Templates:** Create a standard template for each module type (e.g., feature module, data module, core module). This helps teams create modules quickly and correctly.
- **Naming Conventions:** Establish a uniform naming convention for modules (e.g., feature_auth, lib_utils).

2. Development Workflow:

- **Branching Strategy:** Use a branching strategy like Git Flow or Trunk-Based Development. Trunk-Based Development works better with modularization, as developers merge their code frequently to the main branch.
- **Pull Request and Code Review:** Create a strong code review process that checks for new dependencies and changes in the module structure.
- **CI/CD:** Create a CI/CD pipeline for each module that automatically checks builds, tests, and code quality.



Esmaeel Moradi

@esmaeel-moradi700



3.Ownership and Responsibility:

- Each module should have a designated owner or team.
- The module owner is responsible for maintenance, code reviews, and resolving issues related to that module.

4.Communication and Education:

- Regularly educate teams on modularization best practices and changes in governance rules.
- Create a communication channel (e.g., Slack) to discuss modularization issues.



Esmaeel Moradi
@esmaeel-moradi700



Module governance helps prevent chaos in a large codebase and ensures that the app grows in a sustainable way.



Esmaeel Moradi
@esmaeeil-moradi700



Create module analytics and monitoring systems for production applications.

For applications that use dynamic modules, monitoring and analyzing them is crucial for understanding user behavior and for continuous optimization.



Esmaeel Moradi
@esmaeeil-moradi700



1. Key Metrics to Monitor:

- **Download Success Rate:** The percentage of successful download requests for each module.
- **Download Time:** The average time required to download each module.
- **Install Rate:** The percentage of modules that are successfully installed after download.
- **Usage Rate:** The percentage of users who download and use each module.



Esmaeel Moradi
@esmaeeil-moradi700



2. Monitoring System Architecture:

- **Data Collection (Client-Side):** Use the Play Core API to get module download statuses. Use a performance analytics library (like Firebase Performance Monitoring) to track download times and an event analytics tool (like Firebase Analytics) to track events related to module usage.
- **Data Ingestion to Backend:** The collected data should be sent to an analytics server (like Firebase or a custom backend).
- **Analysis and Visualization (Dashboard):** Use tools like Looker Studio or Grafana to create the following dashboards:
 - **Module Health Dashboard:** Shows the download and install success rates for each module.
 - **Usage Dashboard:** Shows the number of users who use each module.
 - **Optimization Dashboard:** Compares download times for different versions of the app.



Esmaeel Moradi

@esmaeeil-moradi700



3. Identifying and Solving Problems:

- **Performance Tracking:** If a module has a low install rate, this could indicate a problem.
- **Identifying Unnecessary Modules:** If a module has a low usage rate, it could be a candidate for removal or redesign.
- **Improving UX:** If a module has a long download time, you can use prefetching strategies to improve it.



Esmaeeil Moradi
@esmaeeil-moradi700



By creating a monitoring and analytics system, an architect can use data-driven decisions for continuous improvement and a better end product.



Esmaeel Moradi
@esmaeel-moradi700

✨ These comprehensive answers aim to equip you with the knowledge required for Architect-level and Senior-level interviews, and to make informed architectural decisions in your real-world projects. Good luck. ✨



Esmaeeil Moradi

@esmaeeil-moradi700